

Project Report: AI Resume-to-Job Fit Analyzer

1. Cover Page

Project Title: AI Resume-to-Job Fit Analyzer

Domain: Artificial Intelligence & Machine Learning

Author: Adil Sukumar

Reg.No.: 25MIB10056

Objective: To build an ATS simulator that uses NLP to score resume compatibility and identify missing keywords against a job description.

2. Introduction

In today's competitive job market, many applicants are rejected by automated ATS before a human ever reads their resume. This project aims to bridge that gap by providing a local AI tool that allows students and job seekers to evaluate their resumes against specific job descriptions. By utilizing fundamental NLP concepts-specifically Term Frequency-Inverse Document Frequency (TF-IDF) and Cosine Similarity-this tool acts as a transparent ATS, providing actionable feedback (missing keywords) rather than just a pass/fail decision.

3. Problem Statement

Students and job seekers struggle to optimize their resumes for ATS because they lack insight into which specific keywords the system expects based on the job description.

4. Functional Requirements

1. Document Loading: Ability to parse .txt and .pdf files for both the resume and the job description.
2. Text Normalization: Cleaning, lowercasing, and removing stop-words from raw text.
3. Feature Extraction: Computing a custom TF-IDF vocabulary across the input corpus.
4. Similarity Scoring: Calculating Cosine Similarity between the resume vector and the job description vector.
5. Feedback Generation: Identifying and ranking the most critical missing keywords.

Project Report: AI Resume-to-Job Fit Analyzer

5. Non-functional Requirements

1. Performance: Vectorized mathematical operations using NumPy for fast scoring.
2. Security: Local processing of files ensures personal resume data is not sent to external servers.
3. Usability: Simple Command-Line Interface (CLI) with clear execution arguments.
4. Reliability: Fallback error handling if an unsupported file type is provided.
5. Scalability: Can be easily adapted to process batches of resumes against a single job description.
6. Maintainability: Strict adherence to a modular architecture separating concerns.
7. Error Handling: Try-except blocks handling file read errors gracefully.
8. Logging: Comprehensive logging to app_debug.log for execution tracking.

6. System Architecture

A Command-Line Interface takes file paths as input and feeds them to a Data Loader. The Data Loader parses text from either PDF or TXT files, passing the raw string to a Text Cleaning utility. The sanitized strings are converted into mathematical vectors by a custom TF-IDF Extractor. These vectors are passed into a Cosine Similarity Engine, which outputs a score to the Match Evaluator. The Evaluator determines missing keywords and prints the final report to the terminal.

7. Design Decisions & Rationale

Custom TF-IDF implementation: Instead of relying on scikit-learn, implementing TF-IDF using standard Python dictionaries and NumPy arrays demonstrates a deep understanding of the underlying mathematics.

NumPy for Cosine Similarity: Native Python loops are slow for vector operations. NumPy provides optimized, C-level operations for computing the dot product and magnitude.

8. Implementation Details

The project is built entirely in Python. It uses PyPDF2 for reading PDF documents and NumPy for all mathematical transformations. The data flow relies heavily on object-oriented programming principles, encapsulating the similarity logic inside a Model pipeline class.

Project Report: AI Resume-to-Job Fit Analyzer

9. Testing Approach

Unit testing involved creating small, controlled text files where the outcome is predictable. For instance, testing a JD of 'Python SQL' against a Resume of 'Python' correctly yields a ~50-70% match and identifies 'SQL' as the missing keyword.

10. Challenges Faced

PDF Extraction issues: Some PDFs contain complex layouts (tables, columns) which confused the text extractor, adding whitespace randomly. This was resolved by writing a aggressive Regex cleaner.

Zero-Division in Cosine Similarity: If a document was entirely filtered out by stop-words, its magnitude was zero, causing a crash. Added a fallback to return 0.0 in this edge case.

11. Learnings & Key Takeaways

- Learned how automated HR systems actually work under the hood.
- Gained hands-on experience building NLP algorithms from mathematical principles rather than black-box libraries.
- Understood the importance of robust data cleaning before applying ML models.

12. Future Enhancements

- Integrate a large language model (LLM) API to suggest exactly *how* to write the missing keywords into bullet points.
- Build a web GUI using Streamlit or Flask.
- Support Word Documents (.docx).

13. References

1. TF-IDF mathematical definitions: <https://en.wikipedia.org/wiki/Tf-idf>
2. PyPDF2 Documentation: <https://pypdf2.readthedocs.io/>

Project Report: AI Resume-to-Job Fit Analyzer

3. NumPy linear algebra documentation: <https://numpy.org/doc/stable/reference/routines.linalg.html>